# 🔥 The 48-Hour "Capability-First" Learning Framework

> **Goal:** In 48 hours, you don't "know the tech" — you **can use it inside a real system and know when it fails**.

That's real learning.

---

## ⏱️ HOUR 0–2 — Orientation (NO CODING)

### Ask only 5 questions

Write answers in plain English.

1. **What job does this tool do?** (One sentence, max)
2. **What problem does it solve that others don't?**
3. **What are its inputs?**
4. **What are its outputs?**
5. **When should I NOT use it?**

If you can't answer these, you are not ready to code.

⛔ Do **not** read docs yet. ⛔ Do **not** watch long tutorials.

---

## ⏱️ HOUR 2–6 — First Contact (AI-assisted coding)

### Rule: YOU do not write syntax

You prompt AI like this:

> "Write the smallest working example that shows the core job of this tool."

For Playwright, that would be:

- open browser
- go to page
- extract one thing
- close

Run it. Let it break. Fix only what blocks execution.

What you learn here:

- setup pain
- environment issues
- basic mental model

This is already more valuable than tutorials.

---

## ⏱️ HOUR 6–12 — Core Capabilities Only

Now test **only the core actions**.

For any tool, identify **3–5 actions** and do them.

Example (Playwright):

- wait for dynamic content
- handle scroll
- extract multiple items
- click + submit

Do **NOT** explore edge features.

Document:

- what worked
- what failed
- what felt fragile

---

## ⏱️ HOUR 12–18 — Break It On Purpose (MOST IMPORTANT)

Now you **attack the tool**.

You intentionally:

- give bad input
- slow network
- dynamic pages
- missing elements
- unexpected states

Ask:

- Does it crash?

- Does it hang?
- Does it silently fail?

💧 This step creates **non-automatable knowledge**.

Most people skip this. That's why they are replaceable.

---

## ⏱️ HOUR 18–24 — Embed It in a Mini System

Now stop thinking about the tool.

Think **SYSTEM**.

Ask:

> "Where does this sit in a real workflow?"

Example:

- Input → Tool → Output
- Tool as a replaceable module

You build a **tiny pipeline**, like:

```
input → tool → structured output → save/log
```

Even if it's ugly, that's fine.

Now the tool is:

- not magic
- not central
- just a component

---

## ⏱️ HOUR 24–36 — Replaceability Test

Now ask the future-proof question:

> "If this tool disappears tomorrow, what replaces it?"

List:

- 1 alternative
- 1 weaker option

- 1 manual fallback

This ensures you **don't emotionally attach** to tools.

---

## ⏱️ HOUR 36–42 — Teach It Simply (Feynman check)

Explain the tool like this:

> "This tool does X. It's good when Y. It breaks when Z. I use it only in this part of my system."

If you can't explain it **simply**, you don't understand it.

---

## ⏱️ HOUR 42–48 — Create a Reusable Asset

Final step: **lock the learning in**.

Create ONE of these:

- a checklist
- a template
- a wrapper function
- a short README
- a diagram

This turns learning into **leverage**.

---

# 🧠 Why this works (future-proof logic)

- AI automates syntax → you don't rely on syntax
- Tools change → you learn capabilities
- Agents improve → you design systems
- Automation grows → you own failure handling

You are learning **how to think**, not **what to type**.

---

## 🚫 What this framework deliberately avoids

- long courses
- memorization
- certifications
- tutorial hell

Those are **pre-AI habits**.

---

## Final rule (burn this into memory)

> **Learn fast. Learn shallow on syntax. Learn deep on failure and system use.**

---